

4 算法设计与 C 程序实现

4.1 分治法

概念：对于一个规模为 n 的问题，若该问题可以容易地解决则直接解决；否则将其分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解决这些子问题，然后将各子问题的解合并得到原问题的解。

步骤：分解（将原问题分解成一系列子问题）——求解（递归地求解各子问题，若子问题足够小，则直接求解）——合并（将子问题的解合并成原问题的解）。

凡是涉及到分组解决的都是分治法。

在分治法中，因为是分解为若干个**相同的问题**，因此会使用到**递归技术**：在函数运行过程中调用自己。

在**二分查找法**、归并排序等算法里，将待排序序列分解为一个个小序列先排序，后再不断扩大范围合并，就是分治法的原理（**将问题规模缩小**）。

4.2 回溯法

概念：有“通用的解题法”之称，可以系统地搜索一个问题的所有解或任一解。在包含问题的所有解的解空间树中，按照深度优先的策略，从根节点触发搜索解空间树。搜索至任一结点时，总是先判断该结点是否肯定不包含问题的解，如果不包含，则跳过对以该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先的策略进行搜索。

可以理解为先进行深度优先搜索，一直向下探测，当此路不通时，返回上一层探索另外的分支，重复此步骤，这就是回溯，意为先一直探测，当不成功时再返回上一层。

一般用于**解决迷宫类**的问题。

4.3 贪心法

概念：总是做出在当前来说是最好的选择，而并不从整体上加以考虑，它所做的每步选择只是当前步骤的局部最优选择，但从整体来说不一定是最优的选择。由于它不比为了寻找最优解而穷尽所有可能解，因此其耗费时间少，一般可以快速得到满意的解，但得不到最优解。

局部贪心，只针对当前的步骤取最优，而非整体考虑。

判断此类算法，就看算法是否是**每一步都取最优**，并且整体题意没有透露出最终结果是最优的。

4.4 动态规划法

概念：在求解问题中，对于每一步决策，列出各种可能的局部解，再依据某种判定条件，舍弃哪些肯定不能得到最优解的局部解，在每一步都经过筛选，以每一步都是最优解来保证全局是最优解。

本质也是将复杂的问题划分成一个个子问题，与分治法不同的是每个子问题间不是相互独立的，并且不全都相同。

常用于求解具有某种最优性质的问题。

此算法会将大量精力放在前期构造表格上面，其会对每一步，列出各种可能的答案，这些答案会存储起来，最终要得出某个结果时，是通过查询这张表来得到的。动态规划法不但每一步最优，全局也最优。

4.5 解题技巧

算法设计是下午软件设计中最难的题型，主要难在 C 代码填空上，因此文老师的建议是先不解决代码填空题（因为最难），先解决其他外围问题（如时间复杂度、算法技巧、取特殊值计算结果），最后解决代码填空，有助于理解整个题目，技巧如下：

1、代码填空：第一小题，最后解决，并不影响解决其他题目，要理解题目算法原理，才能得出答案。另外近几年的算法设计真题有很大的技巧，即便不理解算法原理也可以推导出答案，要结合算法描述中的公式，以及算法代码中类似的分支，能够发现填空的答案在代码中其中地方已经给出，因为算法原理的相似可以通用，尤其是分治法算法。详见文老师真题详解，详述技巧。

要注意的是，当遇到有最小值或最大值参与比较时，若比较出来比最小值更小，接下来肯定要更新这个最小值以及其下标元素值。当遇到一些条件判断的填空时，要注意对应上下文查看哪些变量是作为控制的。

2、算法、时间复杂度：第二小题，先做，考察何种算法很好分辨，涉及到分组就是分治法，局部最优就是贪心法，整体规划最优就是动态规划法，迷宫类的问题是回溯法，记住关键字很好区分；时间复杂度就是看 C 代码中的 for 循环层数和每一层的循环次数，涉及到二分必然有 $O(\log n)$ 。

3、特殊值计算：第三小题，一般应该先做，不需要根据 C 代码，直接根据题目给出的算法原理，一步步推导即可得出答案，耐心推导并不难。但要注意，如果遇到算法原理十分复杂的，建议放弃，掌握问题 1 和 2 的技巧即可。

详细示例及应用见文老师课程视频，有详细技巧的应用讲解。

下午：算法设计与 C 程序实现

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答写在答题纸的对应栏内。

【说明】

用两台处理机 A 和 B 处理 n 个作业。设 A 和 B 处理第 i 个作业的时间分别为 a_i 和 b_i 。由于各个作业的特点和机器性能的关系，对某些作业，在 A 上处理时间长，而对某些作业在 B 上处理时间长。一台处理机在某个时刻只能处理一个作业，而且作业处理是不可中断的，每个作业只能被处理一次。现要找出一个最优调度方案，使得 n 个作业被这两台处理机处理完毕的时间（所有作业被处理的时间之和）最少。

算法步骤：

(1) 确定候选解上界为 R 短的单台处理机处理所有作业的完成时间 m，

$$m = \min \left(\sum_{i=1}^n a_i, \sum_{i=1}^n b_i \right)。$$

(2) 用 $p(x, y, k) = 1$ 表示前 k 个作业可以在 A 用时不超过 x 且在 B 用时不超过 y 时间内处理完成，
则 $p(x, y, k) = p(x - a_k, y, k - 1) \vee p(x, y - b_k, k - 1)$ ($\vee$ 表示逻辑或操作)。

(3) 得到最短处理时间为 $\min(\max(x, y))$ 。

【C 代码】

下面是该算法的 C 语言实现。

(1) 常量和变量说明

n : 作业数

m : 候选解上界

a : 数组，长度为 n ，记录 n 个作业在 A 上的处理时间，下标从 0 开始

b : 数组，长度为 n ，记录 n 个作业在 B 上的处理时间，下标从 0 开始

k : 循环变量

p : 三维数组，长度为 $(m+1) \times (m+1) \times (n+1)$

$temp$: 临时变量

max : 最短处理时间

(2) C 代码

```
#include <stdio.h>
int n, m;
int a[60], b[60], p[100][100][60];
void read() { /*输入 n、a、b，求出 m，代码略*/ }
void schedule() { /*求解过程*/
    int x, y, k;
    for(x = 0; x <= m; x++) {
        for(y = 0; y <= m; y++) {
            (1);
            for(k = 1; k <= n; k++)
                p[x][y][k] = 0;
        }
    }
    for(k = 1; k <= n; k++) {
        for(x = 0; x <= m; x++) {
            for(y = 0; y <= m; y++) {
                if(x - a[k - 1] >= 0) (2);
                if((3)) p[x][y][k] = (p[x][y][k] || p[x][y - b[k - 1]] || p[x - a[k - 1]][y][k]);
            }
        }
    }
}
```



```
        111[k - 1]);  
    }  
}  
}  
void write(){ /*确定最优解并输出*/  
    int x,y, temp,max = m;  
    for(x = 0;x <= m;x++){  
        for (y = 0;y <= m;y++){  
            if( (4) ){  
                temp = (5) ;  
                if(temp < max)    max = temp;  
            }  
        }  
    }  
    printf("\n%d\n", max);  
}  
void main(){    read();    schedule();    write();    }
```

【问题 1】（9 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）～（5）。

【问题 2】（2 分）

根据以上 C 代码，算法的时间复杂度为（6）（用 O 符号表示）。

【问题 3】（4 分）

考虑 6 个作业的实例，各个作业在两台处理机上的处理时间如表 4-1 所示。该实例的最优解为（7），最优解的值（即最短处理时间）为（8）。最优解用 $(x_1, x_2, x_3, x_4, x_5, x_6)$ 表示，其中若第 i 个作业在 A 上处理，则 $x_i=1$ ，否则 $x_i=2$ 。如 $(1, 1, 1, 1, 2, 2)$ 表示作业 1, 2, 3 和 4 在 A 上处理，作业 5 和 6 在 B 上处理。

表 4-1

	作业1	作业2	作业3	作业4	作业5	作业6
处理机 A	2	5	7	10	5	2
处理机 B	3	8	4	11	3	4

答案：

问题 1：

(1) $p[x][y][0] = 1$

(2) $p[x][y][k] = p[x - a[k - 1]][y][k - 1]$

(3) $y - b[k - 1] \geq 0$

(4) $p[x][y][n] == 1$ 或 $p[x][y][n]$ 或 $p[x][y][n] != 0$

(5) $(x \geq y) ? x : y$

问题 2： $O(m^2n)$

问题 3: (7)(1,1,2,2,1,1) (8)15

解析:

问题 1 考察算法设计, `schedule` 函数计算 `p` 数组中元素的值。第一个三重 `for` 循环进行 `p` 数组的初始化, 当 $k=0$ 时, $p[i][j][k]=1$; 而当 k 不等于 0 时, $p[i][j][k]=0$, 因此(1)处应填 “ $p[x][y][0]=1$ ”。在接下来的三重 `for` 循环中, 自底向上计算 $p[i][j][M]$ 的值, 计算根据题中定义来进行, (2)处应填 “ $p[x][y][k]=p[x-a[k-l]][y][k-l]$ ”。对应于上一行的条件, (3)处应填 “ $y-b[k-1]>=0$ ”。`write` 函数中, 已经计算出 `p` 数组元素的值, 因此此时每一个 $p[i][j][n]=1$ 表示一个可行解, 因次(4)处应填 “ $p[x][y][n]=1$ ” 或等价的表达式。其中该解对应的处理时间为 i 和 j 中的较大值, 因此(5)处应填 “ $(x>=y)?x:y$ ”。在所有的可行解中, 确定最小的处理时间即为问题的最优解。

问题 2 考察时间复杂度, 根据上述 C 代码, 函数 `schedule` 有两处三重 `for` 循环, 时间复杂度为 $O(m^2 \cdot n)$ 。

问题 3 是特殊值代入, 根据题意和算法, 可以得到若作业 1、2、5 和 6 在处理机 A, 而作业 3 和 4 在处理机 B 上处理, 可以得到最优解。此时在处理机 A 上的处理时间为 14, 而在处理机 B 上的处理时间为 15, 因此最优解的值, 即最短的处理时间为 15, 而最优解为 (1,1,2,2,1,1)。

[试题 4]

阅读下列说明和 C 代码, 回答下列问题。

[说明]

设有 n 个货物要装入若干个容量为 C 的集装箱以便运输, 这 n 个货物的体积分别为 $\{s_1, s_2, \dots, s_n\}$, 且有 $s_1 \leq c (1 \leq i \leq n)$ 。为节省运输成本, 用尽可能少的集装箱来装运这 n 个货物。

下面分别采用最先适宜策略和最优适宜策略来求解该问题。

最先适宜策略(`firstfit`)首先将所有的集装箱初始化为空, 对于所有货物, 按照所给的次序, 每次将一个货物装入第一个能容纳它的集装箱中。

最优适宜策略(`bestfit`)与最先适宜策略类似, 不同的是, 总是把货物装到能容纳它且目前剩余容量最小的集装箱, 使得该箱子装入货物后闲置空间最小。

[C 代码]

下面是这两个算法的 C 语言核心代码。

(1)变量说明

`n`: 货物数

`C`: 集装箱容量

`s`: 数组, 长度为 n , 其中每个元素表示货物的体积, 下标从 0 开始

b: 数组，长度为 n，b[i]表示第 i+1 个集装箱当前已经装入货物的体积，下标从 0 开始

i,j: 循环变量

k: 所需的集装箱数

min: 当前所用的各集装箱装入了第 i 个货物后的最小剩余容量

m: 当前所需要的集装箱数

temp: 临时变量

(2)函数 firstfit

```
int firstfit() {  
    int i, j;  
    k = 0;  
    for(i = 0; i < n; i++) {  
  
        for(i = 0; i < n; i++){  
            _____(1)_____;  
            while(C - b[j] < s[i]){  
                j++;  
            }  
            _____(2)_____;  
            k = k > (j+1) ? k : (j+1);  
        }  
        return k;  
    }  
}
```

(2)函数 bestfit

```
int bestfit() {
    int i, j, min, m, temp;
    k = 0;
    for(i = 0; i < n; i++) {
        b[i] = 0;
    }
    for(i = 0; i < n; i++) {
        min = C;
        m = k + 1;
        for(j = 0; j < k + 1; j++) {
            temp = C - b[j] - s[i];
            if(temp > 0 && temp < min){
                (3);
                m = j;
            }
        }
        (4);
        k = k > (m+1) ? k : (m+1);
    }
    return k;
}
```

[问题 1]

根据[说明]和[C 代码]，填充 C 代码中的空(1)-(4)处。

[问题 2]

根据[说明]和[C 代码]，该问题在最先适宜和最优适宜策略下分别采用了__(5)___和__(6)___算法设计策略，时间复杂度分别为__(7)___和__(8)___(用 O 符号表示)。

[问题 3]

考虑实例 $n=10$, $C=10$, 各个货物的体积为{4, 2, 7, 3, 5, 4, 2, 3, 6, 2}。该实例在最先适宜和最优适宜策略下所需的集装箱数分别为__(9)___和__(10)___。考虑一般的情况，这两种求解策略能否确保得到最优解?__(11)___(能或否)。

答案:

问题 1:: (1) $j = 0$ (2) $b[j] = b[j] + s[i]$

(3) $min = temp$ (4) $b[m] = b[m] + s[i]$

问题 2: (5) 贪心 (6) 贪心 (7) $O(n^2)$ (8) $O(n^2)$

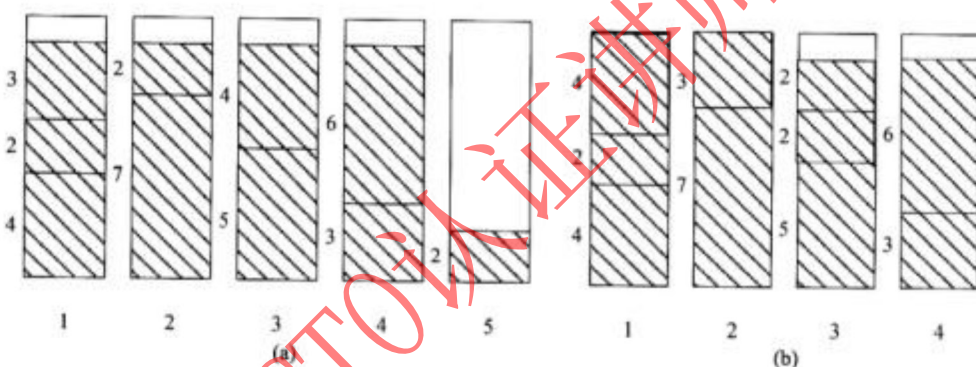
问题 3:: (9) 5 (10) 4 (11) 否

解析:

问题 1 考察算法设计，根据最先适宜算法思想，每取出一个货物，从第一个集装箱开始判断该货物是否能放入集装箱，若能则放入，因此空(1)填 $j = 0$ 。while 循环判断，若货物不能放入集装箱，则考虑下一个集装箱。不满足 while 循环中的条件，说明货物能放入集装箱，因此空(2)填 $b[j] = b[j] + S[i]$ 。根据最优适宜算法思想，每取出一个货物，从第一个集装箱开始，确定能放入该货物且剩余容量最小的集装箱，并把该货物放入该集装箱中。if 条件判断，若找到了比能放入货物且剩余容量更小的集装箱，则剩余容量最小值改为当前的集装箱的剩余容量，因此空(3)填 $min = temp$ 。确定了集装箱后，把货物装入集装箱中，空(4) $b[m] = b[m] + s[i]$ 。

问题 2 考察算法分类和时间复杂度，最先适宜算法总是把货物放入第一个能放入的集装箱，最优适宜算法总是把货物放入能容纳该货物且剩余容量最小的集装箱，因此都是基于贪心策略进行的。函数 firstfit 中的 for 循环考虑 n 个货物，其中嵌套了 while 循环，最多的集装箱数为 n ，因此时间复杂度为 $O(n^2)$ 。函数 bestfit 中的 for 循环考虑 n 个货物，其中嵌套了 for 循环检查每个集装箱的剩余容量，最多的集装箱数为 n ，因此时间复杂度为 $O(n^2)$ 。

问题 3 考察特殊值代入，对实例 $n = 10, C = 10, S = \{4, 2, 7, 3, 5, 4, 2, 3, 6, 2\}$ ，根据最先适宜和最优适宜算法，其具体的装箱方案分别如下图(a)和(b)所示。这两种贪心策略不能确保得到最优解



试题四

设有 m 台完全相同的机器运行 n 个独立的任务，运行任务 i 所需要的时间为 t_i ，要求确定一个调度方案是完成所有任务所需要的时间最短。

假设任务已经按照其运行时间从大到小排序，算法基于最长运行时间作业优先的策略；按顺序先把每个任务分配到一台机器上，然后将剩余的任务依次放入最先空闲的机器。

[C 代码]

下面是算法的 C 语言实现。

(1) 常量和变量说明

m:机器数。

n:任务数。

t[]: 输入数组，长度为 n，其中每个元素表示任务的运行时间，下标从 0 开始。

s[][]:二维数组，长度为 m*n，下标从 0 开始，其中元素 s[i][j]表示机器 i 运行的任务 j 的编号。

d[]: 数组，长度为 m 其中元素 d[i]表示机器 i 的运行时间，下标从 0 开始。

count[]:数组，长度为 m，下标从 0 开始，其中元素 count[i]表示机器 i 运行的任务数。

i: 循环变量。

j: 循环变量。

k: 临时变量。

max: 完成所有任务的时间。

min: 临时变量。

(2) 函数 schedule

```
void schedule(){  
    int i,j,k,max=0  
    for(i=0;i<m;i++){  
        d[i]=0;  
        for(j=0;j<n;j++){  
            s[i][j]=0;  
        }  
    }  
}
```

```
    }  
  }  
  for(i=0;i<m;i++){          //分配前m个任务  
    s[i][0]=j;  
    <u>    (1)    </u> ;  
    count[i]=1;  
  }  
  for(<u>    (2)    </u> i<n,i++){    //分配后n-m个任务  
    int min=d[0];  
    k=0;  
    for(j=1;j<m;j++){          //确定空闲机器  
      if(min>d[j]){  
        min=d[j];  
        k=j;                    // 机器K空闲  
      }  
    }  
    <u>    (3)    </u> ;  
    count[k]=count[k]+1;  
    d[k]=d[k]+t[i];  
  }  
  for(i=0;i<m;i++){          //确定完成所有任务需要的时间  
    if(<u>    (4)    </u>){  
      max=d[i];  
    }  
  }  
}
```

问题 1：根据说明和 C 代码，填充 C 代码中的空(1)-(4)。

问题 2：根据说明和 C 代码，该问题采用了_(5)_算法设计策略，时间复杂度为_(6)_(用 O 符号表示)

问题 3：考虑实例 $m=3$ (编号 0~2), $n=7$ (编号 0~6), 各任务的运行时间为{16,14,6,5,4,3,2}。则在机器 0、1 和 2 上运行的任务分别为_(7)____、_(8)____和_(9)____(给出任务编号)。从任务开始运行到完成所需要的时间为_(10)____。

答案：

问题 1：(1) $d[i]=t[i]$ (2) $i=m$ (3) $s[k][count[k]]=i$ (4) $max<d[i]$

问题 2：(5)贪心 (6) $O(m*n)$

问题 3：(7) 0 (8) 1,5 (9) 2,3,4,6 (10) 17

解析：

问题 1 考察算法设计，解析如下：

根据题中说明和代码注释，算法首先初始化数组d和s中的元素；然后将m个任务分配到m台机器上，此时将任务0, 1, ..., m-1分别分配到机器0, 1, ..., m-1上，同时设置d、s和count数组中的相关元素的值，故空格 (1)填写d[i] = t[i];接下来将剩下的n-m个任务分配到m台机器上，从任务m开始，因此空格 (2)填写i=m，确定首先空闲的机器k，将当前尚未分配的第一个任务分配到机器k上，并设置d、s和count数组中的相关元素的值，故空格 (3)填写s[k][count[k]] = i;最后确定从任务开始到结束所需要的时间，从所有机器的运行时间中选择运行时间最长的机器的运行时间，即最大的d[i]，因此空格 (4)填写max<d[i]。

问题 2 考察算法及时间复杂度，

根据上述C代码，算法有两处两重循环，时间复杂度为O(mn)，有两处一重循环，时间复杂度为O(m)，因此整个算法的时间复杂度为O(mn)。

问题 3 考察特殊值代入运行，解析如下：

根据说明中的算法思想和C代码，首先将任务0、1和2分配到机器0、1和2上运行，运行时间分别为16、14和6。由于任务2的时间最短，故任务3在机器2上运行，机器2上的运行时间为6+5=11，仍然是时间最短，任务4继续在机器2上运行，机器2上的运行时间为11+4=15。此时机器1上的运行时间最短，任务5在机器1上运行，机器1上的运行时间为14+3=17。此时机器2上的运行时间最短，任务6在机器2上运行，机器2上的运行时间为15+2=17。所以任务分配完成。根据此分配，在机器0上运行任务0,运行时间为16;机器1上运行任务1和5,运行时间为17;在机器2上运行任务 2, 3, 4和6,运行时间为17。因此从任务开始到结束的时间为17。

试题四：

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某工程计算中要完成多个矩阵相乘(链乘)的计算任务。

两个矩阵相乘要求第一个矩阵的列数等于第二个矩阵的行数，计算量主要由进行乘法运算的次数决定。采用标准的矩阵相乘算法，计算 $A_m \times n \times B_n \times p$ ，需要 $m \times n \times p$ 次乘法运算。

矩阵相乘满足结合律，多个矩阵相乘，不同的计算顺序会产生不同的计算量。以矩阵 $A_{110 \times 100}$, $A_{2100 \times 5}$, $A_{35 \times 50}$ 三个矩阵相乘为例，若按 $(A_1 \times A_2) \times A_3$ 计算，则需要进行 $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ 次乘法运算;若按 $A_1 \times (A_2 \times A_3)$ 计算，则需要进行 $100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$ 次乘法运算。可见不同的计算顺序对计算量有很大的影响。

矩阵链乘问题可描述为：给定 n 个矩阵，矩阵 A_i 的维数为 $p_{i-1} \times p_i$ ，其中 $i = 1, 2, \dots, n$ 。确定一种乘法顺序，使得这 n 个矩阵相乘时进行乘法的运算次数最少。

由于可能的计算顺序数量非常庞大，对较大的 n，用蛮力法确定计算顺序是不实际的。经过对问题进行分析，发现矩阵链乘问题具有最优子结构，即若 $A_1 \times A_2 \times \dots \times A_n$ 的一个最优计算顺序从第 k 个矩阵处断开，即分为 $A_1 \times A_2 \times \dots \times A_k$ 和 $A_{k+1} \times A_{k+2} \times \dots \times A_n$ 两个子问题，则该最优解应该包含 $A_1 \times A_2 \times \dots \times A_k$ 的一个最优计

算顺序和 $A_{k+1} * A_{k+2} * \dots * A_n$ 的一个最优计算顺序。据此构造递归式，

$$\text{cost}[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \text{cost}[i][k] + \text{cost}[k+1][j] + p_i * p_{k+1} * p_{j+1} & \text{if } i < j \end{cases}$$

其中， $\text{cost}[i][j]$ 表示 $A_{i+1} * A_{i+2} * \dots * A_{j+1}$ 的最优计算的计算代价。最终需要求解 $\text{cost}[0][n-1]$ 。

【C 代码】

算法实现采用自底向上的计算过程。首先计算两个矩阵相乘的计算量，然后依次计算 3 个矩阵、4 个矩阵、...、n 个矩阵相乘的最小计算量及最优计算顺序。下面是算法的 C 语言实现。

(1)主要变量说明

n: 矩阵数

seq[]: 矩阵维数序列

cost[][]: 二维数组，长度为 $n * n$ ，其中元素 $\text{cost}[i][j]$ 表示 $A_{i+1} * A_{i+2} * \dots * A_{j+1}$ 的最优计算的计算代价

trace[][]: 二维数组，长度为 $n * n$ ，其中元素 $\text{trace}[i][j]$ 表示 $A_{i+1} * A_{i+2} * \dots * A_{j+1}$ 的最优计算对应的划分位置，即 k

(2)函数 cmm

```
#define N 100
int cost[N][N];
int trace[N][N];
int cmm(int n, int seq[]){
    int tempCost;
    int tempTrace;
    int i, j, k, p;
    int temp;
    for(i = 0; i < n; i++){ cost[i][i] = 0; }
    for(p = 1; p < n; p++){
        for(i = 0; (1); i++){
            (2);
            tempCost = -1;
            for(k = i; k < j; k++){
                temp = (3);
                if(tempCost == -1 || tempCost > temp){
                    tempCost = temp;
                    (4);
                }
            }
            cost[i][j] = tempCost;
            trace[i][j] = tempTrace;
        }
    }
    return cost[0][n - 1];
}
```

【问题 1】(8 分)

根据以上说明和 C 代码，填充 C 代码中的空(1)~(4)。

【问题 2】(4 分)

根据以上说明和 C 代码，该问题采用了 (5) 算法设计策略，时间复杂度 (6)。(用 O 符号表示)

【问题 3】(3 分)

考虑实例 $n=6$ ，各个矩阵的维数：A1 为 5×10 ，A2 为 10×3 ，A3 为 3×12 ，A4 为 12×5 ，A5 为 5×50 ，A6 为 50×6 ，即维数序列为 5,10,3,12,5,50,6。则根据上述 C 代码得到的一个最优计算顺序为 (7) (用加括号方式表示计算顺序)，所需要的乘法运算次数为 (8)。

答案：

问题 1: (1) $i < n - p$ (2) $j = i + p$ (3) $\text{cost}[i][k] + \text{cost}[k+1][j] + \text{seq}[i] * \text{seq}[k+1] * \text{seq}[j+1]$

(4) $\text{tempTrace} = k$

问题 2: (5) 动态规划 (6) $O(n^3)$

问题 3: (7) $((A1A2)((A3A4)(A5A6)))$ (8) 2010

解析:

问题 1 考察算法实现，C 程序中主要部分是三重循环，循环变量 p 定义了求解问题的规模，因为是从自底向上，因此， p 的值应该是从 1 到 $n-1$ ，即从规模为 1 的问题一直到规模为 $n-1$ 的问题。循环变量 i 是要求解的子问题的起始，从 0 开始，最大为 $n-p-1$ ，故 (1) 处应填 $i < n-p$ 。确定了 i 和 p 之后，下来就要确定 j 了，显然，空 (2) 处为 $j=i+p$ 。循环变量 k 是问题 $A_i * A_{i+1} * \dots * A_j$ 的划分位置，对每一个 k ，都要计算需要的计算成本，可以根据递归式来填写，空 (3) 处为 $cost[i][k] + cost[k+1][j] + seq[i] * seq[k+1] * seq[j+1]$ 。确定每个问题 $A_i * A_{i+1} * \dots * A_j$ 的划分位置 k 之后，要把这个 k 值记住，放在变量 $tempTrace$ 中，即空 (4) 处填写 $tempTrace=k$ 。

问题 2 考查算法的设计策略和时间复杂度，从题干说明可以很容易看出，问题具有最优子结构和重叠子问题，采用自底向上的方法求解，这些都是动态规划的典型特点，因此采用的是动态规划设计策略。从上述 C 程序很容易分析出，程序中没有递归，存在三重循环，故时间复杂度为 $O(n^3)$ 。

问题 3 考查算法的应用。通过一个具体实例可以更容易理解问题和求解方法。可以根据问题 1 中的程序执行来求解。启发式的思路是先把维度最大的消掉，如 $A5 * A6$ 相乘之后，维度 50 就没有了，所以考虑这两个矩阵先相乘；然后是 $A3 * A4$ 相乘之后，维度 12 就没有了，所以考虑这两个矩阵相乘；接着， $A1 * A2$ 相乘之后，维度 10 就没有了，所以考虑这两个矩阵相乘。这样可以确定相乘的顺序 $((A1A2)((A3A4)(A5A6)))$ ，需要的计算开销分别是 $5 * 50 * 6 = 1500$, $3 * 12 * 5 = 180$, $5 * 10 * 3 = 150$, $3 * 5 * 6 = 90$, $5 * 3 * 6 = 90$ ，把上述值相加，即 $1500 + 180 + 150 + 90 + 90 = 2010$ 。

试题四

采用归并排序对 n 个元素进行递增排序时，首先将 n 个元素的数组分成各含 $n/2$ 个元素的两个子数组，然后用归并排序对两个子数组进行递归排序，最后合并两个已经排好序的子数组得到排序结果。下面的 C 代码是对上述归并算法的实现，其中的常量和变量说明如下：

arr: 待排序数组

p, q, r : 一个子数组的位置为从 p 到 q ，另一个子数组的位置为从 $q+1$ 到 r

begin, end: 待排序数组的起止位置

left, right: 临时存放待合并的两个子数组

$n1, n2$: 两个子数组的长度

i, j, k : 循环变量

mid: 临时变量

【C 代码】

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 65536
void merge(int arr[],int p,int q,int r) {
    int *left,*right;
    int n1,n2,i,j,k;
    n1=q-p+1;
    n2=r-q;
    if((left=(int*)malloc((n1+1)*sizeof(int)))==NULL){
        perror("malloc error");
        exit(1);
    }
    if((right=(int*)malloc((n2+1)*sizeof(int)))==NULL){
        perror("malloc error");
        exit(1);
    }
    for(i=0;i<n1;i++){
        left[i] = arr[p+i];
    }
    left[i] = MAX;
    for(i=0;i<n2;i++){
        right[i] = arr[q+i+1];
    }
    right[i]= MAX;
    i=0; j=0;
    for (k=p; (1) k++ ) {
        if(left[i]> right[j]){
            (2)
            j++;
        }else{
            arr[k]= left[i];
            i++;
        }
    }
}

void mergeSort(int arr[],int begin,int end){
    int mid;
    if ( (3) ){
        mid=(begin+ end)/2;
        mergeSort(arr, begin, mid);
        (4)
        merge(arr, begin, mid, end);
    }
}
```

问题 4.1 根据以上说明和 C 代码，填充 C 代码中的空(1)~(4)。

问题 4.2 根据题干说明和以上 c 代码，算法采用了(5)算法设计策略。分析时间复杂度时，列出其递归

式为(6)，解得渐进时间复杂度为(7)（用 O 符号表示）。空间复杂度为(8)（用 O 符号表示）。

问题 4.3 两个长度分别为 n_1 和 n_2 的已排好序的子数组进行归并，根据上述 C 代码，则元素之间比较次数为(9)。

答案：

问题 1: (1) $k \leq r$ 或 $k < r+1$ (2) $\text{arr}[k] = \text{right}[j]$
(3) $\text{begin} < \text{end}$ (4) $\text{mergeSort}(\text{arr}, \text{mid}+1, \text{end})$

问题 2: (5) 分治 (6) $T(n) = 2T(n/2) + O(n)$ 或 $T(n) = 2T(n/2) + f(n)$ ($f(n)$ 为线性函数)
(7) $O(n \log n)$ (8) $O(n)$

问题 3: (9) $n_1 + n_2$

解析：

问题 1 考察算法填空，归并排序是一种经典的算法，其基本思想是把 n 个元素构成的数组分成两个 $n/2$ 个元素构成的子数组，再进一步划分，一直到每个子数组仅包含 1 个元素，此时再把两两有序的数组合并成更大的有序数组，一直到整个数组有序为止。题目 C 程序中两个函数，`merge` 函数将两个有序数组合并成一个更大的有序数组。归并过程是首先将两个有序的子数组的元素分别放到 `left` 和 `right` 数组中，然后依次比较这两个数组中的元素，从小到大把元素放到 `arr` 数组的特定元素中。包含空格(1)的 `for` 循环中，给出了将 `left` 和 `right` 元素放入 `arr` 中，放入的位置是从 `p` 到 `r`，因此，空格 (1) 填写 $k \leq r$ 。在比较 `left[i]` 和 `right[j]` 元素时，若 `left[i] > right[j]`，则应该把 `right[j]` 的值放入 `arr[k]` 中，因此空格 (2) 填写 $\text{arr}[k] = \text{right}[j]$ 。`mergeSort` 函数进行数组的排序。若数组元素个数大于 1，则继续划分，因此空格 (3) 填写 $\text{begin} < \text{end}$ 。将数组从 `arr[begin]` 到 `arr[end]` 划分为 `arr[begin]` 到 `arr[mid]` 和 `arr[mid + 1]` 到 `arr[end]` 两个部分，因此空格(4)填写 $\text{mergeSort}(\text{arr}, \text{mid} + 1, \text{end})$ 。

问题 2 考察算法策略和复杂度，归并排序算法是一个典型的分治算法。每次将一个规模为 n 的问题变成两个规模为 $n/2$ 的子问题，划分时直接从中间分开，因此划分采用 $O(1)$ 的时间，然后是递归求解两个子问题，根据 `merge` 函数代码，合并的时间是线性时间 $O(n)$ 。因此递归式为 $T(n) = 2T(n/2) + f(n)$ ， $f(n)$ 为线性函数。用主方法求解，得到时间复杂度为 $O(n \log n)$ 。由于在归并过程中，需要 `left` 和 `right` 两个辅助数组，其规模为待排序的数组长度，即 $O(n)$ 。

问题 3 中元素之间的比较次数就是 `merge` 函数的最后一个 `for` 循环体执行的次数，由于 k 从 p 到 r ，故循环体执行次数为 $r - p + 1$ 次，即 $n_1 + n_2$ 次。

试题四：

阅读下列说明和C代码，回答问题1至问题3，将解答写在答题纸的对应栏内。

【说明】

计算一个整数数组a的最长递增子序列长度的方法描述如下：

假设数组a的长度为n，用数组b的元素b[i]记录以a[i](0≤i<n)为结尾元素的最长递增子序列的长度，则数组a的最长递增子序列的长度为 $\max_{0 \leq i < n} \{b[i]\}$ ；其中b[i]满足最优子结构，可递归定义为：

$$\begin{cases} b[0] = 1 \\ b[i] = \max_{\substack{0 \leq k < i \\ a[k] < a[i]}} \{b[k]\} + 1 \end{cases}$$

【C代码】

下面是算法的C语言实现。

(1) 常量和变量说明

a：长度为n的整数数组，待求其最长递增子序列

b：长度为n的数组，b[i]记录以a[i](0≤i<n)为结尾元素的最长递增子序列的长度，其中0≤i<n

len：最长递增子序列的长度

ij：循环变量

temp：临时变量

(2) C程序

```
#include <stdio.h>
int maxL(int*b, int n) {
    int i, temp=0;
    for(i=0; i<n; i++) {
        if(b[i]>temp)
            temp=b[i];
    }
    return temp;
}
int main() {
    int n, a[100], b[100], i, j, len;
    scanf("%d", &n);
    for(i=0; i<n; i++) {
        scanf("%d", &a[i]);
    }
    (1) ;
    for(i=1; i<n; i++) {
        for(j=0, len=0; (2) ; j++) {
            if((3) && len<b[j])
                len=b[j];
        }
        (4) ;
    }
    Printf("len:%d\n", maxL(b,n));
    printf("\n");
}
```

问题1 根据说明和C代码，填充C代码中的空(1)～(4)。

问题2 根据说明和C代码，算法采用了(5)设计策略，时间复杂度为(6) (用O符号表示)。

问题 3 已知数组 $a=\{3,10,5,15,6,8\}$ ，根据说明和 C 代码，给出数组 b 的元素值。

答案：

问题 1: (1) $b[0]=1$ (2) $j<i$ (3) $a[j]<=a[i]$ (4) $b[i]=len+1$

问题 2: (5) 动态规划法 (6) $O(n^2)$

问题 3: $b=\{1,2,2,3,3,4\}$

解析：

问题 1 补充算法语句，根据题干描述，用一个数组 b 来记录数组 a 每个子数组的最长递增子序列的长度，即 $b[i]$ 记录 $a[0..i]$ 的最长递增子序列的长度。首先，只有一个元素的数组的最长递增子序列的长度为 1，即给 $b[0]$ 直接赋值 1。因此，空 (1) 处填写 “ $b[0] = 1$ ”。两重 for 循环中，第一重是从 a 数组的第二个元素开始，考虑每个子数组 $a[0..i]$ 的最长递增子序列的长度，第二重是具体的计算过程。考虑子数组 $a[0..i]$ ，其最长递增子序列的长度应该等于子数组 $a[0..i-1]$ 中的比元素 $a[i]$ 小的元素的最长递增子序列的长度加 1，当然，可能存在多个元素比元素 $a[i]$ 小，那么存在多个最长递增子序列的长度，此时，取最大者。因此，空处填写 “ $j<i$ ”，即考虑子数组 $a[0..i-1]$ 。空 (3) 处填写 “ $a[j]<=a[i]$ ”，要求元素值小于等于 $a[i]$ 而且目前的长度应该小于当前考虑的子数组的最长子序列长度。空 (4) 处填写 “ $b[i] = len+1$ ”。

问题 2 可以首先完成，从题干说明和 C 程序来看，这是一个最优化问题，而且问题具有最优子结构，一个序列的最长递增子序列由其子序列的最长递增子序列构成。在计算过程中采用了自底向上的方式来进行，这具有典型的动态规划特征。因此采用的是动态规划设计策略。C 程序中，有两重 for 循环，因此时间复杂度为 $O(n^2)$ 。

问题 3 直接将输入代入计算，输入数组为数组 $a = \{3,10,5, 15,6, 8\}$ ，很容易得到，子数组 $a[0..0]$, $a[0..1]...a[0..5]$ 的最长递增子序列的长度分别为 1,2,2,3,3,4,因此答案为 $b=\{1,2,2,3,3,4\}$ 。

下午：面向对象的程序设计与实现-C++

试题五

(共 15 分，在试题五和试题六中任选 1 题解答，如果两题都解答，则视试题五的解答有效。)

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某咖啡店当卖咖啡时，可以根据顾客的要求在其中加入各种配料，咖啡店会根据所加入的配料来计算费用。咖啡店所供应的咖啡及配料的种类和价格如下表所示。